Strukture podataka (120)

Analiza složenosti algoritama

Sadržaj

- Algoritmi
- Analiza složenosti algoritma
  - T(N) složenost algoritma
  - Određivanje reda algoritma
  - Problem maksimalne sume podniza

Algoritmi

- Algoritam je jasno određen niz (jednostavnih) naredbi koje treba izvršiti odgovarajućim redoslijedom kako bi se riješio zadani problem.
- Analiza složenosti algoritma definira koliko efikasno taj algoritam rješava dani problem.
- Analiza složenosti algoritama omogućava:
  - procjenu vremena izvršavanja » smanjenje vremena izvršavanja;
  - memoriju potrebnu za izvršavanje algoritma.



Primjer pretraživanja

- Za sortirani niz X od n članova potrebno je pronaći član čija je vrijednost A?
- Sekvencijalno pretraživanje:

```
int br = 0;
while (br < n && X[br] != A)
    br ++;
```

- Binarno pretraživanje:

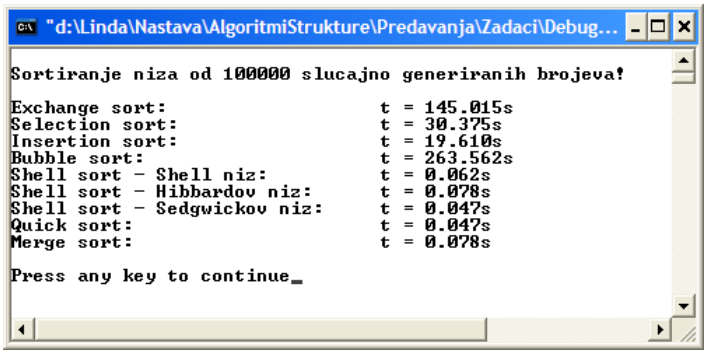
```
int p = 0, k = n - 1, s;
int br = 0;
while (p < k && br == 0)
{
    s = (p+k)/2;
    if( A == X [s] ) br = s;
    else if (A < X[s]) k = s -1;
    else p = s +1;
}
```

- Ako se traži broj veći od svih koji postoje u nizu?

	Broj usporedbi	
Veličina niza	Sekvencijalno	Binarno
128	128	8
1 024	1 024	11
1 048 576	1 048 576	21
4 294 967 296	4 294 967 296	33



Primjer sortiranja



Analiza složenosti algoritma

- Kako bi analiza bila moguća potrebno je poznavati model izvođenja algoritma uz slijedeće pretpostavke i ograničenja:
  - Model je standardno računalo kod kojeg se naredbe izvršavaju sekvencijalno;
  - Model sadrži standardan niz jednostavnih naredbi kao što su zbrajanje, množenje, usporedba i sve te operacije traju jednako;
  - Ne određuje se stvaran broj ciklusa procesora (ovisi o tipu procesora);
  - Ne broje se sve naredbe koje se izvršavaju jer to ovisi o programskom jeziku u kojem je algoritam pisan.
- Nedostatci ovakvog pristupa:
  - Kod stvarnih računala različite operacije ne traju jednako;
  - Zanemaruje se učinak brze i virtualne memorije …

Analiza složenosti algoritma

- Efikasnost algoritma tj. vrijeme izvođenja algoritma se definira kao **broj** ponavljanja **osnovnih operacija** ovisno o **broju ulaznih parametara**!!!!

- *Osnovne operacije* – u algoritmu se odabere određena naredba ili grupa naredbi tako da je ukupno vrijeme izvršavanja algoritma proporcionalno s brojem ponavljanja te naredbe ili grupe naredbi.
  - Ignoriraju se kompleksni detalji (uvećavanje petlje ili pokazivača).

- *Broj ulaznih parametara* ovisi o algoritmu i tipu podataka:
  - jednodimenzionalni nizovi – broj elemenata;
  - dvodimenzionalni nizovi – broj redaka * broj stupaca.



Analiza složenosti algoritma

- Odabir osnovne operacije ovisi o tome što se analizira.

- U primjeru algoritma za sekvencijalno pretraživanje:

```
int br = 0;                          // * - osnovne operacije
while (br < n && X[br] != A)         // * - operacije koje se ne smatraju
        br ++;                       //         osnovnima
```

- Kod sortiranja zamjenom (exchange sort) mogu npr. postojati dvije osnovne operacije:

```
for(i = 0; i< n-1; i++)
    for(j = i+1; j<n; j++)
        if (x[i] < x[j])                    // usporedba vrijednosti
                zamjena(x[i], x[j]);        // zamjena
```



Analiza složenosti algoritma

- Vrijeme izvršavanja algoritma ovisi o:
  - broju ulaznih parametara:
    - Primjer: Zbrajanje elemenata niza:
      - Osnovna operacija se uvijek radi isti broj puta za isti N.
    - Definira se **T(N)** – "every-case" složenost algoritma – mjera koliko se puta izvršava osnovna operacija za slučaj N ulaznih parametara.
  - o vrijednostima ulaznih parametara:
    - Primjer: Sekvencijalnog pretraživanja:
      - ako se traži element niza koji je prvi u nizu – o. o. se radi samo 1;
      - ako se traži element koji ne postoji u nizu – o. o. se radi N puta.
    - W(n) – "worst-case" sl. alg, definira maksimalni br. izvršavanja o. op;
    - A(n) – "average-case" sl. alg, definira prosječan br. izvršavanja o. op;
    - B(n) – "best-case" sl. alg, najmanji mogući br. izvršavanja o. operacije.



T(n) – "every-case" složenost algoritma

- **T(n)** – definira se kao broj izvršavanja osnovne operacije za ***n*** ulaznih parametara, u slučaju kada broj izvršavanja ne ovisi o vrijednostima ulaznih parametara.

- **Primjer 1:** Zbrajanje **n** elemenata niza **X**:

```
sum = 0;                        // 1. definirati osnovnu operaciju
for(int i = 0; i < n; i++)      // 2. definirati broj ulaznih parametara
  sum += X[i];
```

  - osnovna operacija: uvećavanje sum
  - broj ulaznih parametara: n
- **Zaključak:** bez obzira na vrijednosti elemenata niza, osnovna operacija će se izvršiti n puta:

  T(n) = n



T(n) – "every-case" složenost algoritma

- **Primjer 2:** Sortiranje zamjenom n elemenata niza x

```
for(i = 0; i< n-1; i++)
        for(j = i+1; j<n; j++)
            if (x[i] < x[j])
                    zamjena(x[i], x[j]);
```

  - osnovna operacija: usporedba x[i] i x[j]
  - broj ulaznih parametara: n
- U ovom slučaju treba odrediti koliko ima prolazaka kroz for petlje:
  - for (i) => n-1 prolazak
  - for(j):
    - 1. prolazak kroz for(i) uzrokuje n-1 prolazak kroz for(j)
    - 2. prolazak kroz for(i) uzrokuje n-2 prolazak kroz for(j)
    - …
- **Zaključak:**

  T(n) = (n-1) + (n-2) + (n-3) + … + 1=(n-1)*n/2



T(n) – "every-case" složenost algoritma

- **Primjer 3:** Množenje matrica

```
for(int i = 0; i< n; i++)
        for(int j = 0; j<n; j++) {
            C[i][j] = 0;
            for (int k = 0; k< n; k++)
                    C[i][j] += A[i][k]*B[k][j];
    }
```

  - osnovna operacija: računanje C[i][j] elementa u for(k) petlji
  - broj ulaznih parametara: n redaka i n stupaca
- U ovom slučaju treba odrediti koliko ima prolazaka kroz for petlje:
  - uvijek ima n prolazaka kroz for (i) petlju:
    - svaki prolazak kroz for(i) uzrokuje n prolazaka kroz for(j) petlju
    - svaki prolazak kroz for(j) uzrokuje n prolazaka kroz for(k) petlju
- **Zaključak:**

  T(n) = n*n*n = n^3



Osnovna pravila za računanje T(n)

1. Vrijeme izvršavanja for petlje je vrijeme izvršavanja osnovnih operacija unutar petlje, pomnoženo s brojem ponavljanja petlje.

2. Ugniježđene for petlje se analiziraju od unutra prema vani. Ukupno vrijeme izvršavanja osnovnih naredbi unutar petlji je jednako vremenu izvršavanja naredbi pomnoženom s brojem izvršavanja svih petlji.

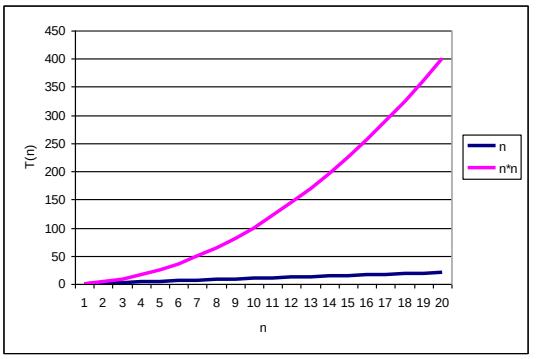
3. Vremena izvođenja uzastopnih petlji se zbrajaju.

4. Vrijeme izvođenja if/else izraza je jednako vremenu izvođenja uvjeta uvećanom za vrijeme izvođenja bloka koji traje duže.



Primjena teoretskog razmatranja

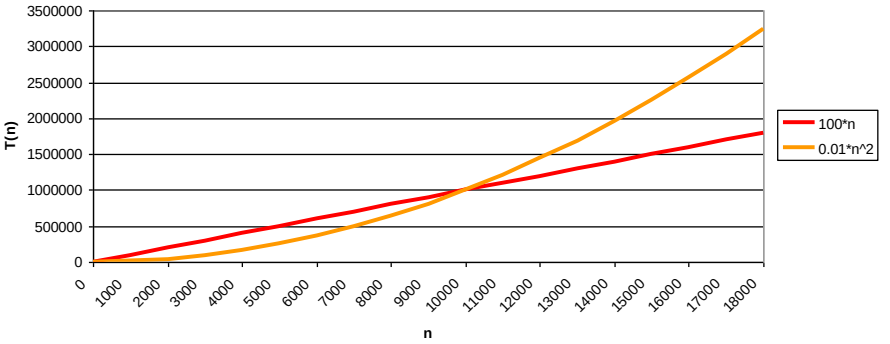
- Ako se neki algoritam riješi na dva različita načina i za jedan je T(n) = n, a za drugi T(n) = n^2, koji je algoritam efikasniji?





Primjena teoretskog razmatranja

- A za T(n) = 100*n, a za drugi T(n) = 0.01*n^2, koji je algoritam efikasniji?



- Interesantan je relativni porast vrijednosti, a ne trenutna vrijednost.
- Bilo koji linearni algoritam će u konačnici biti efikasniji od bilo kojeg kvadratnog algoritma, jer se u teoretskoj analizi algoritama promatra "krajnje" ponašanje algoritma.

Određivanje reda ("ordrer") algoritama

- T(n) = 0.1n^2 + n + 100

n	0.1n^2	0.1n^2+n+100
10	10	120
20	40	160
50	250	400
100	1 000	1 200
1000	100 000	101 100

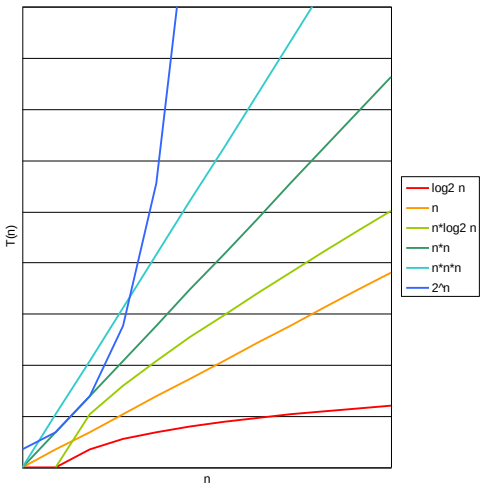
- Tablica pokazuje da kvadratni član u konačnici dominira funkcijom.
- Zbog toga je moguće odbaciti sve članove nižeg reda i označiti funkciju kao čistu kvadratnu.
- Niz svih funkcija složenosti koje se mogu opisati čistom kvadratnom funkcijom se naziva $\Theta(n^2)$ i ta funkcija je n^2 reda.

Određivanje reda algoritama

- Kojem redu pripada algoritam za sortiranje zamjenom?

$$T(n) = \frac{(n-1)*n}{2}$$

$T(n) \in \Theta(n^2)$



- Neke od najčešćih kategorija složenosti su:
  - $\Theta(\log_2 n)$
  - $\Theta(n)$
  - $\Theta(n*\log_2 n)$
  - $\Theta(n^2)$
  - $\Theta(n^3)$
  - $\Theta(2^n)$



Određivanje reda algoritama

- T(n) daje više informacija o vremenu izvršavanja algoritma nego samo poznavanje reda algoritma (Θ), ali se Θ koristi jer je ponekad vrlo teško odrediti T(n) i važnije je poznavati relativni porast vrijednosti nego njenu trenutnu vrijednost.

- Za detaljniju definiciju reda algoritma koriste se
  - $O(f(n))$ – "big O" predstavlja gornju asimptotu funkcije, tj. opisuje kako će se funkcija složenosti algoritma ponašati u najgorem slučaju.
    - $o(f(n))$ – "little o" predstavlja gornju asimptotu funkcije s još strmijim porastom
  - $\Omega(f(n))$ – predstavlja donju asimptotu funkcije , tj. opisuje kako će se funkcija složenosti algoritma ponašati u najboljem slučaju.



Određivanje reda algoritama

- Svojstva reda:
  1. Ako je b > 1, a > 1 tada
     $\log_a n \in \Theta(\log_b n)$
     Sve logaritamske funkcije složenosti su istog reda i on se predstavlja sa $\Theta(\log_2 n)$.
  2. Ako je b > a > 0 tada
     $a^n \in o(b^n)$
     Sve eksponencijalne funkcije složenosti nisu istog reda.
  3. Za sve a > 0
     $a^n \in o(n!)$, podrazumijeva da je n! lošiji od bilo koje eksp. f-je.
  4. $\Theta(\log_2 n)$ $\Theta(n)$ $\Theta(n \log_2 n)$ $\Theta(n^2)$ $\Theta(n^j)$ $\Theta(n^k)$ $\Theta(a^n)$ $\Theta(b^n)$ $\Theta(n!)$
     za k > j > 2 i b > a > 1
     Ako je g(n) u kategoriji lijevo od f(n) tada je $g(n) \in o(f(n))$.



Problem maksimalne sume podniza

- Za niz brojeva A_0, A_1, A_2, … A_{N-1} (mogu biti i pozitivni i negativni) potrebno je odrediti maksimalnu sumu podniza.

- Npr. za niz 4, -3, 5, -2, -1, 2, 6, -2
  maksimalna suma podniza je 11 (A_0 do A_7) ili

$$Max\sum_{k=i}^{j} A_k = 11,\ i = 0\ \ j = 7$$

- Osnovne operacije:
  - Proračun sume
  - Usporedba



Problem maksimalne sume podniza

- **Algoritam 1**:

```
    int MaxSumaPodniza1(int A[], int N) {
/*1*/       int trenSuma, maxSuma = 0, i, j, k;
/*2*/       for(i = 0; i < N; i++)
/*3*/           for(j = i; j < N; j++)
                {
/*4*/               trenSuma = 0;
/*5*/               for(k = i; k<=j; k++)
/*6*/                   trenSuma += A[k];
/*7*/               if(trenSuma > maxSuma)
/*8*/                   maxSuma = trenSuma;
                }
/*9*/       return maxSuma;
    }
```

- Koliki je red izvođenja ovog algoritma?
  - $\Theta(N^3)$



Problem maksimalne sume podniza

- **Algoritam 2**:

```
    int MaxSumaPodniza2(int A[], int N)
    {
            int trenSuma, maxSuma, i, j;
            maxSuma = 0;
/*1*/       for(i = 0; i < N; i++)
            {
/*2*/           trenSuma = 0;
/*3*/           for(j = i; j < N; j++)
                {
/*4*/               trenSuma += A[j];
/*5*/               if(trenSuma > maxSuma)
/*6*/                   maxSuma = trenSuma;
                }
            }
            return maxSuma;
    }
```

- Red ovog algoritma je $\Theta(N^2)$.



Problem maksimalne sume podniza

- **Algoritam 3**:
  - Rekurzivni algoritam za kojeg vrijedi Θ(N*logN);
  - Koristi "divide-and-conquer" pristup, tj. ideja je da se osnovni problem razbije na dva približno jednaka dijela koja se zatim rješavaju rekurzivno.
  - Kod ovog problema maksimalna suma podniza se može nalaziti na jednom od tri mjesta:
    - lijevoj polovini niza – rješava se rekurzivno;
    - desnoj polovini niza – rješava se rekurzivno;
    - u sredini i obuhvaća obije polovine – pronađe se najveća suma u lijevoj polovini - počevši od zadnjeg elementa te polovine i najveća suma desne polovine - koja počevši od prvog elementa te polovine.



Problem maksimalne sume podniza

- **Algoritam 3:** primjer za niz: 4, -3, 5, -2, -1, 2, 6, -2

lijeva polovina	desna polovina
4 -3 5 -2	-1 2 6 -2

  - Maks. suma podniza s lijeve strane je 6 (A_0 - A_2)
  - Maks. suma podniza s desne strane je 8 (A_5 - A_6)
  - Maks. suma lijevog podniza koja uključuje zadnji element podniza je 4 (A_0 - A_3)
  - Maks. suma desnog podniza koja uključuje prvi element podniza je 7 (A_4 - A_6)
  - To znači da je maksimalna suma podniza 11 koja se dobije zbrajanjem elemenata (A_0 - A_6).



Problem maksimalne sume podniza

- **Algoritam 3**:

```
int MaxSumaPodniza3(int A[], int L, int D) {
        int maxLSum, maxDSum, maxLGranSum, maxDGranSum, lGranSum, dGranSum, sredina, i;
/* 1*/          if(L == D)
/* 2*/                  if(A[L] > 0)  return A[L];
/* 3*/                  else          return 0;
/* 4*/          sredina = (L + D)/2;
/* 5*/          maxLSum = MaxSumaPodniza3(A, L, sredina);
/* 6*/          maxDSum = MaxSumaPodniza3(A, sredina+1, D);
/* 7*/          maxLGranSum = lGranSum = 0;
/* 8*/          for(i = sredina; i >= L; i--) {
/* 9*/                  lGranSum += A[i];
/*10*/                  if(lGranSum  > maxLGranSum) maxLGranSum = lGranSum;
                }
/*11*/          maxDGranSum = dGranSum = 0;
/*12*/          for(i = sredina + 1; i <= D; i++) {
/*13*/                  dGranSum += A[i];
/*14*/                  if(dGranSum  > maxDGranSum) maxDGranSum = dGranSum;
                }
/*15*/          return Max3(maxLSum, maxDSum, maxLGranSum + maxDGranSum);
        } // osnovna operacija je samo proračun sume
```



Problem maksimalne sume podniza

- **Algoritam 3 – Analiza algoritma**:
  - Za N = 1 => T(1) = 1 (linije od 1-3)
  - Za niz veličine N vrijeme potrebno da se riješi problem je T(N):
    - 2 for petlje (linije 8-10, 12-14)
      - Zajedno obrađuju sve elemente od A_0 – A_{N-1} => $\Theta(N)$
    - dodjela vrijednosti (linije 4, 7, 11, 15)
      - Zanemarive u usporedbi s for petljama pa se ne računa
    - 2 rekurzivna poziva (linije 5 i 6)
      - Šalje se upola manje elemenata pa vrijedi => T(N/2) tj. 2*T(N/2)
  - T(N) = 2*T(N/2) + N
    - T(1) = 1
    - T(2) = 2*T(1) + 2 = 4
    - T(4) = 2*T(2) + 4 = 12
    - T(8) = 2*T(4) + 8 = 32
    - T(16) = 2*T(8) + 16 = 80

$T(N) = N*\log_2 N + N$

$T(N) = N*\log_2 N$

$\Rightarrow \Theta(N*\log_2 N)$



Problem maksimalne sume podniza

- **Algoritam 4**:

```
        int MaxSumaPodniza4(int A[], int N)
        {
                int maxSum, trenSum, i;
/* 1*/          maxSum = trenSum = 0;
/* 2*/          for(i = 0; i<N; i++)
                {
/* 3*/                  trenSum += A[i];
/* 4*/                  if(trenSum > maxSum) maxSum = trenSum;
/* 5*/                  else if(trenSum < 0) trenSum = 0;
                }
/* 6*/          return maxSum;
        }
```

- $T(N) \in \Theta(N)$



Ispit - primjer

- Za donje primjere napravi analizu vremena izvođenja T(N) i odrediti red Θ(T(N)) ako je osnovna operacija uvećanje sume.
  -
```
sum = 0;
for(i = 0; i<N; i++)
    sum ++;
```
  -
```
sum = 0;
for(i = 0; i < N; i++)
    for(j= 0; j < N; j++)
        sum++;
```
  -
```
sum = 0;
for(i = 0; i < N; i++)
    for(j= 0; j < N*N; j++)
        sum++;
```
  -
```
sum = 0;
for(i = 0; i < N; i++)
    for(j= 0; j < i; j++)
        sum++;
```
  -
```
sum = 0;
for(i = 0; i <N; i++)
    for(j= 0; j < i*i; j++)
        for(k = 0; k < j; k++)
            sum++;
```



Analiza složenosti algoritama

Kraj